```
Task: Display SupplierID and the number of products supplied by
each supplier.
Show suppliers with the highest number of products first.


select *
from `e1.products`
limit 5

select
  SupplierID,
  count(*) as no_of_products
from `e1.products`
group by SupplierID
order by no_of_products desc



Task: Calculate the total sales value for each ProductID using
the Total column from order_items.
Display the products from highest to lowest sales value and show
only the top 10.


select *
from `e1.order_items`
limit 5
```

```sql
select
  ProductID,
  sum(Total) as total_sales,
from `e1.order_items`
group by ProductID
order by total_sales desc
limit 10
```

Task: Display SupplierID and the number of products supplied by each supplier. Show suppliers with the highest number of products first.

Task: Calculate the total sales value for each ProductID using the Total column from order_items. Display the products from highest to lowest sales value and show only the top 10.

Task: For only payments where Status = 'Success', calculate the number of transactions and total payment amount for each Method. Display methods with the highest total payment amount first.

Task: Calculate the number of orders placed by every CustomerID. Display only customers who have placed more than 3 orders.

Task: Consider only products having an MRP greater than ₹20,000. For each CategoryID, count the number of such products. Display only categories having more than 2 qualifying products.

Task: For each WarehouseID, calculate:

Total stock
Average stock per inventory record
Maximum stock in an inventory record
Minimum stock in an inventory record

Sort warehouses by total stock from highest to lowest.

Task: For every Status in the orders table, calculate the number of orders. Display the most common order status first.

```
Task: Consider only order-item records where SellingPrice >
10000. For each ProductID, calculate:


Number of order-item records
Total quantity sold
Total sales value
Average selling price


Display only products whose total sales value is greater than
₹50,000,
sorted by total sales value from highest to lowest.


select
  ProductID,
  sum(Total) as total_sales
from `e1.order_items`
group by ProductID
having total_sales > 50000
order by total_sales desc




Task: For only payments where Status = 'Success',
```

```
calculate the number of transactions and total payment amount
for each Method.
Display methods with the highest total payment amount first.


select *
from `e1.payments`
limit 5

select
  count(*) as no_of_transactions,
  sum(amount) as total_payment
from `e1.payments`
where Status = "Success"
group by Method
order by total_payment desc



Task: Calculate the number of orders placed by every CustomerID.
Display only customers who have placed more than 3 orders.


select *
from `e1.orders`
limit 5


select
  CustomerID,
```

```
  count(*) as no_of_orders
from `e1.orders`
group by CustomerID
having no_of_orders > 3
order by no_of_orders desc




Task: Consider only products having an MRP greater than ₹20,000.
For each CategoryID, count the number of such products.
Display only categories having more than 2 qualifying products.



select *
from `e1.products`
limit 5

select
  CategoryID,
  count(*) as no_of_products
from `e1.products`
where MRP > 900
group by CategoryID
having no_of_products > 2
order by no_of_products desc


Task: For every WarehouseID, calculate:
```

```
Total stock
Average stock per inventory record
Maximum stock
Minimum stock

Sort warehouses by total stock from highest to lowest.

select *
from `e1.inventory`
limit 5



select
  WarehouseID,
  sum(stock) as total_stock,
  round(avg(stock), 2) as avg_stock,
  max(stock) as max_stock,
  min(stock) as min_stock
from `e1.inventory`
group by WarehouseID
order by total_stock desc




Task: For every Status in the orders table, calculate the number
of orders.
Display the most common order status first.
```

```sql
select
  status,
  count(*) as no_of_orders
from `e1.orders`
group by Status
order by no_of_orders desc


Task: Consider only order_items records where SellingPrice >
10000.
For each ProductID, calculate:

Number of order-item records
Total quantity sold
Total sales value
Average selling price

Display only products whose total sales value is greater than
₹50,000, sorted by total sales value from highest to lowest.


select
  count(*) as no_of_records,
  sum(Quantity) as total_qty,
  sum(Total) as total_sales,
  avg(SellingPrice) as avg_sp
from `e1.order_items`
where SellingPrice > 10000
group by ProductID
having total_sales > 50000
```

```
order by total_sales desc
```